

Session 16

Quantum Bayes

Session road-map

0. Recap: why probabilistic reasoning after QGNNs
1. Two tracks for “Quantum Bayes” (A: quantum speedups for estimation; B: quantum probabilistic models)
2. From probabilities to circuits (q-sampling prep) — correct mapping & controlled conditionals
3. Tiny quantum Bayes nets (2–3 nodes) — sampling and QAE examples (Qiskit 2.x code)
4. Inference with Quantum Amplitude Estimation (QAE): theory, iterative/MLE variants, NISQ pragmatics
5. Quantum Bayesian networks (qBNs): minimal NISQ view vs research view
6. Uncertainty quantification (UQ) & hybrid loops (how to integrate QAE into a classical Bayes workflow)
7. Honest comparison: classical vs quantum Bayes (where advantage might appear)
8. Practical guidance & pitfalls (angles, controls, transpilation, mitigation)
9. Worked toy example: Cloudy → Rain → WetGrass (3-node net) with code + experiments to run

0) Recap: why Bayes, why quantum?

- **Why Bayes?** Many ML tasks demand calibrated uncertainty: healthcare, safety, decision making under scarce data. Bayesian methods quantify uncertainty via posteriors; often require many probability/expectation queries (Monte Carlo, VI, MCMC).
- **Why quantum?** Circuits can prepare distributions and amplitude-encode probabilities. **Amplitude estimation** (QAE) gives a quadratic improvement in sample complexity in the ideal, oracle model: $O(1/\epsilon)$ oracle calls vs $O(1/\epsilon^2)$ classical samples to reach additive error ϵ . On NISQ, iterative or MLE amplitude-estimation variants reduce circuit depth and are more practical.

Bottom line: Two useful, distinct uses: (A) use quantum subroutines to accelerate probability/expectation estimation for a classical Bayesian workflow; (B) build quantum probabilistic models (qBNs) where states/channels represent distributions — promising but earlier stage.

1) Two meanings of “Quantum Bayes”

(A) Quantum-accelerated Bayesian inference (practical path)

- Keep classical BN or statistical model. For expensive probability/expectation queries (normalisers, large integrals), build an oracle A that prepares a flag qubit whose amplitude encodes the event of interest, then run QAE (iterative/MLE variants) to estimate the probability with fewer oracle calls asymptotically. Combine quantum estimates with classical Bayes updates.

(B) Quantum Bayesian networks (qBNs) / quantum probabilistic models (research)

- Nodes are quantum registers (pure/mixed states); edges = CPTP maps or controlled unitaries encoding conditional dependence. The joint is a density operator; inference $\rightarrow$ measurement + channel composition. Includes QCBMs, Born machines, quantum Boltzmann machines. Rich semantics (non-factorizable Born distributions) but small-scale and conceptually different from classical Bayes.

2) From probabilities to circuits (q-sampling prep) — exact mapping

Single Bernoulli event

To prepare a qubit whose measurement in Z yields $\Pr(1) = p$, use an R_Y rotation with angle θ satisfying:

$$\Pr(1) = \sin^2\left(\frac{\theta}{2}\right) = p \quad \Rightarrow \quad \theta = 2 \arcsin(\sqrt{p}).$$

Thus

$$R_Y(\theta) |0\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + \sin\left(\frac{\theta}{2}\right) |1\rangle = \sqrt{1-p} |0\rangle + \sqrt{p} |1\rangle.$$

Measuring in Z returns $|1\rangle$ with probability p . (Important: **use** $\sqrt{p}$ inside arcsin.)

Conditional Bernoulli (parent $\rightarrow$ child)

Suppose $X \rightarrow Y$ with prior $\Pr(X = 1) = p_X$ and conditionals $p_1 = \Pr(Y = 1 \mid X = 1)$, $p_0 = \Pr(Y = 1 \mid X = 0)$.

Circuit:

1. Prepare X : $R_Y(2 \arcsin \sqrt{p_X})$ on qubit q_0 .
2. Prepare Y controlled on X :
 - Apply controlled- $R_Y(2 \arcsin \sqrt{p_1})$ with control q_0 (applies when $X=1$).
 - Apply controlled-on-0 rotation for p_0 : implement control-on-0 via X sandwich:
 X on control, controlled- $R_Y(2 \arcsin \sqrt{p_0})$, X on control.

This yields the joint distribution on (X,Y) when measuring both qubits.

3) Tiny quantum Bayes nets (worked code) — Qiskit 2.x

Below are two practical templates: (A) sampling baseline with `Sampler` (good for NISQ), (B) iterative amplitude estimation (IAE) using Qiskit's amplitude estimation algorithm variants (less depth than textbook QAE).

Note: APIs evolve. The code below uses `qiskit_aer.primitives.Sampler` / `Estimator` for local sim and `qiskit.algorithms` amplitude estimation classes where available. If your local Qiskit version exposes slightly different class names, the conceptual calls are the same.

```
1 # pip install qiskit qiskit-aer
2 import numpy as np
3 from qiskit import QuantumCircuit
4 from qiskit_aer.primitives import Sampler
```

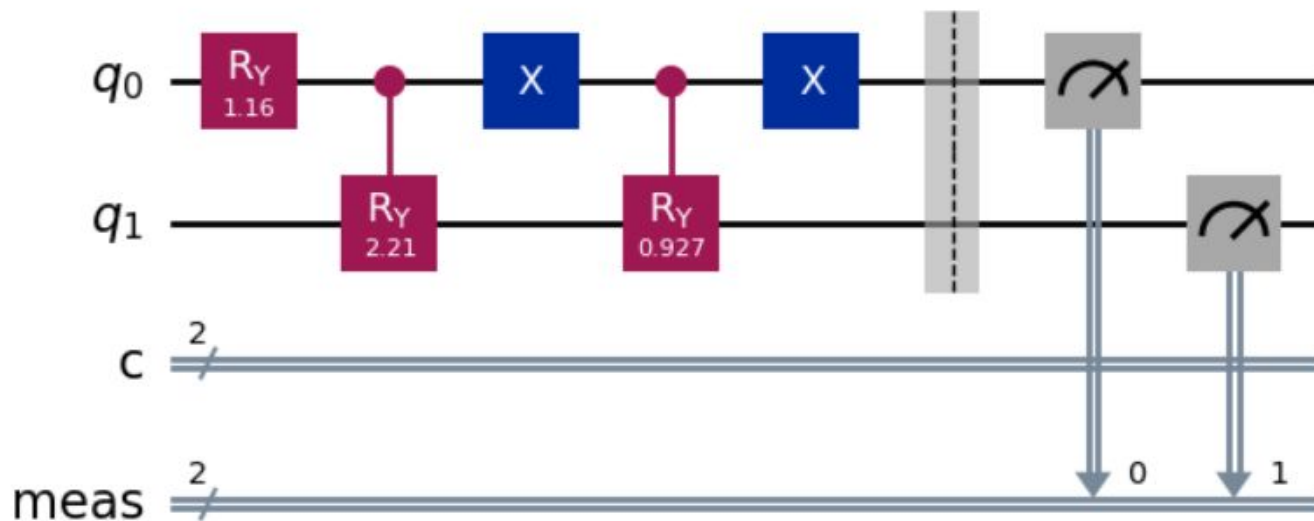
```
1 # model parameters
2 pX = 0.3
3 pY_given_X1 = 0.8
4 pY_given_X0 = 0.2
5
6 # angles (note sqrt)
7 theta_x = 2*np.arcsin(np.sqrt(pX))
8 theta_y1 = 2*np.arcsin(np.sqrt(pY_given_X1))
9 theta_y0 = 2*np.arcsin(np.sqrt(pY_given_X0))
```



```

1 qc = QuantumCircuit(2, 2) # q0 = X, q1 = Y
2 qc.ry(theta_x, 0)         # prepare X
3 # controlled RY on q1 controlled by q0 (for X=1)
4 qc.cry(theta_y1, 0, 1)
5 # controlled-on-0: X, controlled-ry, X
6 qc.x(0)
7 qc.cry(theta_y0, 0, 1)
8 qc.x(0)
9 qc.measure_all()
10 qc.draw('mpl')

```





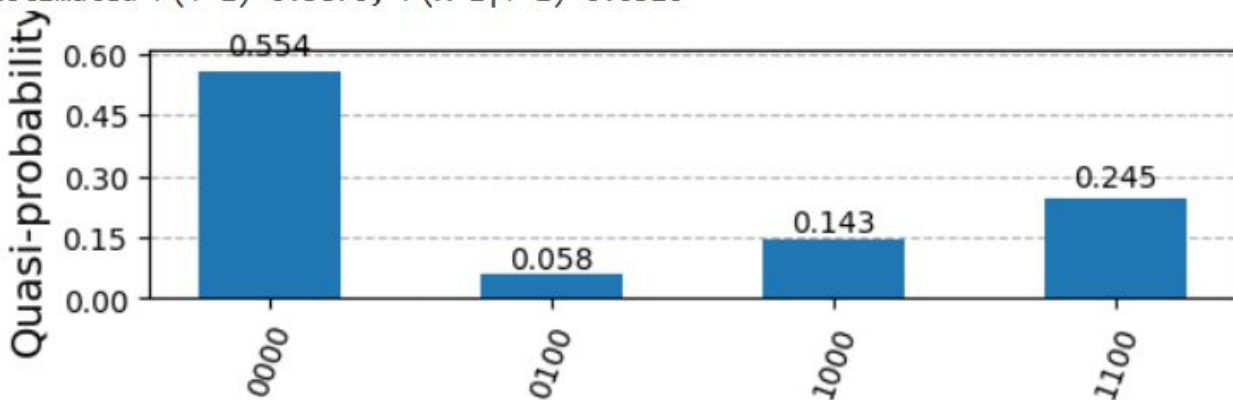
```
1 sampler = Sampler()
2 shots = 20000
3 job = sampler.run([qc], shots=shots)
4 res = job.result()
5 # res.quasi_dists is a list; for single circuit use res.quasi_dists[0]
6 dist = res.quasi_dists[0].nearest_probability_distribution().binary_probabilities()
```

```

1 print(dist) # dict of bitstring -> prob
2 # compute empirical P(Y=1) and P(X=1|Y=1)
3 from collections import defaultdict
4 from qiskit.visualization import plot_histogram
5 counts = defaultdict(float)
6 for bitstring, p in dist.items():
7     # assume bitstring format 'q1q0' where q1=Y, q0=X
8     y = int(bitstring[0]); x = int(bitstring[1])
9     counts[(x,y)] += p * shots
10
11 p_y1 = (counts[(0,1)] + counts[(1,1)]) / shots
12 p_x1_given_y1 = counts[(1,1)] / (counts[(0,1)] + counts[(1,1)])
13 print(f"Estimated P(Y=1)={p_y1:.4f}, P(X=1|Y=1)={p_x1_given_y1:.4f}")
14 plot_histogram(dist, figsize=(6, 2))

```

{'0100': 0.05845, '1000': 0.1426, '1100': 0.24495, '0000': 0.554}
Estimated P(Y=1)=0.3876, P(X=1|Y=1)=0.6320



4) Inference with QAE — theory & NISQ variants

Classic QAE (textbook)

- Uses phase estimation on the Grover operator to estimate amplitude a encoded by the oracle A .
- Asymptotic query complexity: $O(1/\epsilon)$ oracle calls to achieve additive error ϵ versus $O(1/\epsilon^2)$ classical sampling.

NISQ-friendly AE variants

- **Iterative Amplitude Estimation (IAE):** iterative calls with adaptive measurements, no QFT, uses one or few ancilla qubits. Good tradeoff between depth and query savings.
- **Maximum-Likelihood Amplitude Estimation (MLAE):** uses maximum-likelihood estimation on a set of repeated Grover rounds; found to be robust to moderate noise.
- **Median/ratio estimators / Bayesian AE:** other variants with different robustness.

Practical advice

- Build **oracle** A cheaply — if A is expensive to prepare (deep circuits), the QAE advantage disappears.
- AE advantage is in *oracle calls*, not necessarily in wall-clock time on noisy hardware (mitigation and depth add cost).
- Use AE when repeated, high-precision probability queries dominate overall runtime (e.g., inner loop of a nested Monte Carlo).

5) Quantum Bayesian networks (qBNs)

Minimal, NISQ-friendly interpretation (practical)

- Treat each conditional $P(\text{child} \mid \text{parents})$ as a controlled rotation layer (q-sampling style). The whole directed acyclic graph becomes a state-preparation circuit.

Measuring the output qubits yields samples from the joint Born distribution (Born probabilities). You can then perform classical Bayes conditioning on the measured bitstrings.

Learning: fit rotation angles by maximum likelihood (optimize angles to match observed counts).

Limitations:

- Multi-parent conditionals require multi-controlled rotations which transpile to many two-qubit gates; ancilla tricks reduce depth at ancilla cost.
- Conceptual shift: quantum channels / density matrices allow correlations beyond classical factorization; semantics differ from classical BN.

Research view (beyond NISQ)

- Nodes are quantum registers (mixed states), edges are CPTP channels; inference involves composing channels and partial trace. Powerful, but training and interpretability are open research problems.

6) UQ & hybrid inference loops (how to use QAE inside a classical Bayes workflow)

Typical hybrid loop

1. Classical model defines prior & likelihood factors; identify expensive probability / expectation queries needing many samples (e.g., marginal likelihoods or rare-event probabilities).
2. For each such query, build an oracle A that prepares a flag qubit indicating the event. Use IAE/MLAE to estimate probability a to precision ϵ .
3. Compute posterior(s) classically using the quantum-estimated values; repeat if further queries arise.

Examples where this helps

- Estimating expected utility under uncertainty for many candidate decisions (utility queries dominated).
- Posterior predictive checks where many conditional probabilities are needed.
- Nested Monte Carlo where inner integrals can be accelerated.

Practical UQ patterns

- Estimate both point estimates and credible intervals (IAE returns CI approx).
- Combine quantum estimates with classical uncertainty (propagate variance in Bayes updates).
- Validate quantum estimates against Monte Carlo on small problems.

7) Classical vs Quantum Bayes — honest comparison

Aspect	Classical Bayes	Quantum Bayes (NISQ practical)
Probability estimation	Monte Carlo $O(1/\epsilon^2)$	QAE $O(1/\epsilon)$ oracle calls (ideal); IAE/MLAE practical
Model scale	Large BNs, VI, MCMC	Tiny nets (2–6 qubits) feasible today
Robustness	Mature (PyMC, Stan)	Sensitive to noise, readout bias
Oracles	trivial (data sampling)	require building A — may be expensive
Best uses	general inference	inner loops dominated by queries, or where amplitude oracle is cheap

Key: quantum advantage requires cheap/efficient oracles and many high-precision

8) Practical guidance & pitfalls (actionable list)

- **Correct amplitude mapping:** use $\theta = 2 \arcsin(\sqrt{p})$ for $R_Y(\theta)$. Double-check math (common source of bugs).
- **Clamp probabilities:** clamp p to $[10^{-9}, 1 - 10^{-9}]$ before $\arcsin$ to avoid numerical issues.
- **Control-on-0 trick:** implement controlled-on-0 by surrounding control with `x` gates. E.g., to apply controlled-Ry when control==0: `x(control); cry(angle, control, target); x(control)`.
- **Multi-controlled gates:** decompose with ancillas or use `mcx` patterns; check transpiler output and count two-qubit gates — they dominate error.
- **Use iterative/MLE AE on NISQ:** they avoid deep QFT and require fewer qubits; still test on noisy simulator first.
- **Readout mitigation:** essential for small probabilities — use calibration circuits to invert measurement noise.

- **Reuse compiled subcircuits:** build and transpile reusable `A` or `state_prep` once and bind parameters to avoid repeated transpilation costs.
- **Validate:** always compare quantum estimates to classical Monte Carlo on small instances before trusting hardware results.
- **Shot scheduling:** start with low shots during development and increase near convergence/testing.
- **Latency & sessions:** use Runtime sessions (e.g., IBM Runtime) when running many oracle calls on hardware to amortize setup overhead.

9) Worked toy example — Cloudy $\rightarrow$ Rain $\rightarrow$ WetGrass (3-node Bayes net)

Model: $C \rightarrow R \rightarrow W$. Parameters:

- $p_C = \Pr(C = 1)$ (prior cloudiness)
- $p_{R|C=1} = \Pr(R = 1 \mid C = 1), p_{R|C=0} = \Pr(R = 1 \mid C = 0)$
- $p_{W|R=1} = \Pr(W = 1 \mid R = 1), p_{W|R=0} = \Pr(W = 1 \mid R = 0)$

```
1 # params
2 pC = 0.4
3 pR_c1, pR_c0 = 0.6, 0.1
4 pW_r1, pW_r0 = 0.9, 0.2
5
6 def theta(p):
7     return 2*np.arcsin(np.sqrt(p))
8
9 qc = QuantumCircuit(3, 3)# q0=C, q1=R, q2=W
10 qc.ry(theta(pC), 0)
11 # R controlled on C
12 qc.cry(theta(pR_c1), 0, 1)
13 qc.x(0)
14 qc.cry(theta(pR_c0), 0, 1)
15 qc.x(0)
16 # W controlled on R
17 qc.cry(theta(pW_r1), 1, 2)
18 qc.x(1)
19 qc.cry(theta(pW_r0), 1, 2)
20 qc.x(1)
21 qc.measure_all()
```

```

23 sampler = Sampler()
24 shots = 20000
25 job = sampler.run([qc], shots=shots)
26 dist = job.result().quasi_dists[0]
27 bin_probs = dist.nearest_probability_distribution().binary_probabilities()
28 print(bin_probs) # map bitstring->prob
29 plot_histogram(bin_probs, figsize=(6,2))

```

010000': 0.00575, '011000': 0.02625, '101000': 0.03075, '110000': 0.05605, '1

